DATABASE MANAGEMENT SYSTEMS LAB

Lab Report Submitted to

K. R. Mangalam University

for

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

YUKTI VADEHRA, ROLL NO. 2401010128

Course Teacher

Ms. Mansi Kajal

School of Engineering & Technology

EXPERIMENT- 7

Aim

To understand and implement **Transaction Control Language (TCL)** commands such as **ROLLBACK** and **COMMIT** along with table creation, insertion, and updates.

Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

(a) Insert five records
(b) Give Increment of 25% salary for Mathematics Department.
(c) Perform Rollback command
(d) GiveIncrementof15%salaryforCommerce Department
(e) Perform commit command

CREATE DATABASE

CREATE DATABASE school_yukti_db;

Explanation

- CREATE DATABASE is used to create a new database.
- school_yukti_db is the name of the database.

Use Database

USE school_yukti_db;

Explanation

- USE selects the database to perform operations on it.

Create Teacher Table

```
CREATE TABLE Teacher (
  Name VARCHAR(50),
  DeptNo INT,
  DateOfJoining DATE,
  DeptName VARCHAR(50),
  Location VARCHAR(50),
  Salary DECIMAL(10,2)
);
```

Explanation

- CREATE TABLE creates a new table.
- **Fields:**
  - Name → Teacher name (text)
  - DeptNo → Department number (integer)
  - DateOfJoining → Joining date (DATE type)
  - DeptName → Department name
  - Location → Work location
  - Salary → Salary with 2 decimal precision

a. Insert 5 Records

```
INSERT INTO Teacher VALUES
('Yukti', 101, '2020-06-15', 'Mathematics', 'Delhi', 50000),
('Neha ', 102, '2019-03-10', 'Commerce', 'Mumbai', 45000),
('Aadi', 101, '2021-01-20', 'Mathematics', 'Delhi', 55000),
('Mansi', 103, '2018-07-25', 'Science', 'Chennai', 60000),
('Sonal', 102, '2022-02-12', 'Commerce', 'Mumbai', 48000);
```

Explanation

- INSERT INTO adds records into the table.
- Multiple rows inserted in a single query.

Start Transaction

```
START TRANSACTION;
```

Explanation

- Begins a transaction block.
- Changes made after this can be **committed or rolled back**.

b. Increment 25% Salary (Mathematics Department)

```
UPDATE Teacher
SET Salary = Salary + (Salary * 0.25)
WHERE DeptName = 'Mathematics';
```

Explanation

- UPDATE modifies existing records.
- SET defines new value.
- WHERE filters only Mathematics department.

Clause Used

- WHERE → ensures only selected rows are updated

c. Perform ROLLBACK

ROLLBACK;

Explanation

- Cancels all changes made after START TRANSACTION.
- Salary increment of 25% is **undone**.

d. Increment 15% Salary (Commerce Department)

UPDATE Teacher
SET Salary = Salary + (Salary * 0.15)
WHERE DeptName = 'Commerce';

Explanation

- Updates only Commerce department salaries.
- Adds 15% increment

.

e. Perform COMMIT

COMMIT;

Explanation

- Saves all changes permanently.
- After this, changes **cannot be rolled back**.

Display Final Table

SELECT * FROM Teacher;

Explanation

- Displays all records from the table.

Name	DeptNo	DateOfJoining	DeptName	Location	Salary
Yukti	101	2020-06-15	Mathematics	Delhi	50000.00
Neha	102	2019-03-10	Commerce	Mumbai	51750.00
Aadi	101	2021-01-20	Mathematics	Delhi	55000.00
Mansi	103	2018-07-25	Science	Chennai	60000.00
Sonal	102	2022-02-12	Commerce	Mumbai	55200.00

RESULT

The **Teacher table** was successfully created and populated with records. The salary of teachers in the Mathematics department was increased by 25%, but the changes were undone using the **ROLLBACK** command. Then, a 15% salary increment was applied to the Commerce department and permanently saved using the **COMMIT** command. This experiment demonstrated the use of transaction control commands to ensure data consistency and recovery.

EXPERIMENT-8

Aim

To perform operations using **GROUP BY**, **ORDER BY**, and **date functions** in SQL.

Create Database

CREATE DATABASE sales_db;

Explanation

- Creates a new database named yukti_sales_db.

Use Database

USE yukti_sales_db;

Explanation

- Selects the database to work on.

Create Sales Table

```
CREATE TABLE Sales (
  SalesNo INT PRIMARY KEY,
  Salesname VARCHAR(50),
  Branch VARCHAR(50),
  Salesamount DECIMAL(10,2),
  DOB DATE
);
```

Explanation

- SalesNo → Primary Key (unique for each record)
- Salesname → Name of salesman
- Branch → Branch location
- Salesamount → Sales value
- DOB → Date of Birth

(a) Insert 5 Records

```
INSERT INTO Sales VALUES
(1, 'Yukti', 'Delhi', 50000, '1995-12-21'),
(2, 'Neha', 'Mumbai', 60000, '1998-07-15'),
(3, 'Aadi', 'Delhi', 55000, '1992-12-05'),
(4, 'Mansi', 'Chennai', 70000, '1996-03-10'),
(5, 'Sonal', 'Mumbai', 65000, '1994-12-18');
```

Explanation

- Inserts multiple records into the table.

(b) Total Sales Amount in Each Branch

```
SELECT Branch, SUM(Salesamount) AS Total_Sales
FROM Sales
GROUP BY Branch;
```

Branch	Total_Sales
Delhi	105000.00
Mumbai	125000.00
Chennai	70000.00

Explanation

- SUM() → calculates total
- GROUP BY → groups records by branch

(c) Average Sales Amount in Each Branch

```
SELECT Branch, AVG(Salesamount) AS Avg_Sales
FROM Sales
GROUP BY Branch;
```

Branch	Avg_Sales
Delhi	52500.000000
Mumbai	62500.000000
Chennai	70000.000000

Explanation

- AVG() → calculates average
- GROUP BY groups data branch-wise

(d) Salesmen Born in December

```
SELECT Salesname,
DATE_FORMAT(DOB, '%d-%b-%y') AS Formatted_DOB
FROM Sales
WHERE MONTH(DOB) = 12;
```

	Salesname	Formatted_DOB
▶	Yukti	21-Dec-95
	Aadi	05-Dec-92
	Sonal	18-Dec-94

Explanation

- MONTH(DOB) = 12 → filters December births
- DATE_FORMAT() → formats date as:
  - %d → day
  - %b → month (short form like Dec)
  - %y → year (2 digits)

(e) Display Name and DOB in Alphabetical Order of Month

SELECT Salesname, DOB
FROM Sales
ORDER BY MONTHNAME(DOB);

	Salesname	DOB
▶	Yukti	1995-12-21
	Aadi	1992-12-05
	Sonal	1994-12-18
	Neha	1998-07-15
	Mansi	1996-03-10

Explanation

- ORDER BY → sorts data
- MONTHNAME(DOB) → sorts based on month name alphabetically

RESULT

This experiment demonstrates how to:

- Group data using GROUP BY
- Perform calculations using aggregate functions
- Format and filter date values
- Sort records using ORDER BY